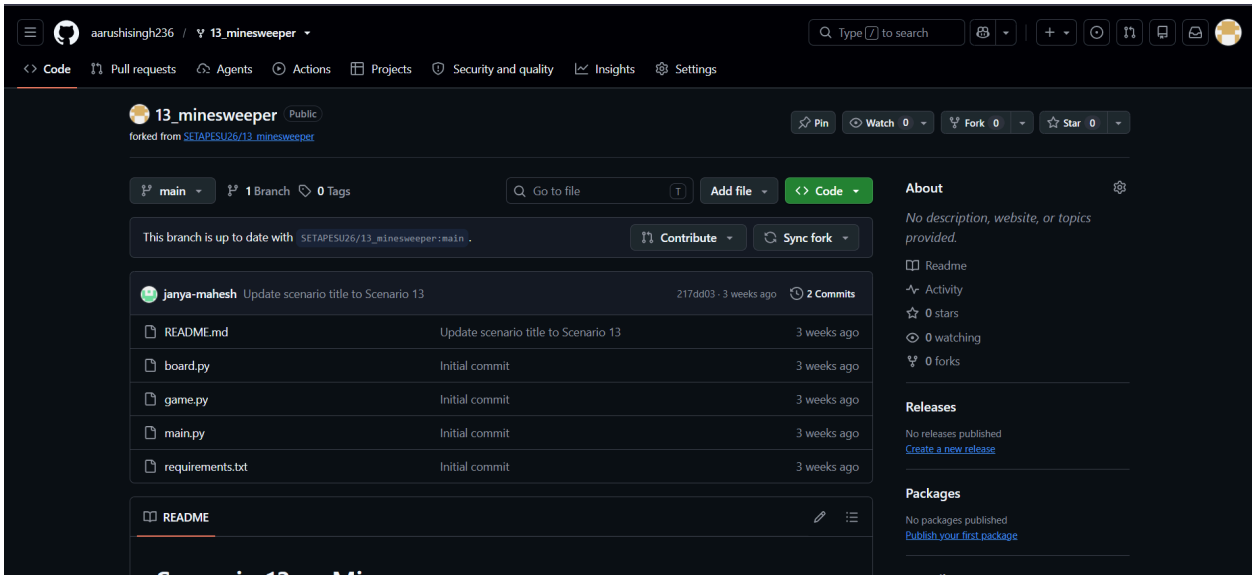
SE LAB

Name: Aarushi Singh
SRN: PES2UG24CS015
Repo Assigned: 13_minesweeper
Personal Repo: Lab 4

Forked:



Original Game:
Initial Execution and Basic Command Testing

The original Minesweeper program was executed successfully using `python main.py`. The game displayed a 6 × 6 board with hidden cells.

The following initial tests were performed:
1. Cell reveal: The command `r 1 1` revealed the top-left cell, which displayed the number 1.
2. Flagging: The command `f 6 6` marked the bottom-right cell with the flag symbol `F`.
3. Invalid coordinates: The command `r 7 7` displayed the message `Outside the board.` without terminating the program.

These tests confirmed that the basic command interface, flagging, and this invalid-coordinate case worked in the original version. Further testing was required to investigate the remaining defects and implement the requested features.

```
PS C:\Users\233285\Downloads\LAB\SE Lab\Solo\13_minesweeper> python main.py
Commands: r row col | f row col | q

   1 2 3 4 5 6
 1 # # # # # #
 2 # # # # # #
 3 # # # # # #
 4 # # # # # #
 5 # # # # # #
 6 # # # # # #
>
```

```
    1 2 3 4 5 6
  1 1 # # # # #
  2 # # # # # #
  3 # # # # # #
  4 # # # # # #
  5 # # # # # #
  6 # # # # # #
 > f 6 6

    1 2 3 4 5 6
  1 1 # # # # #
  2 # # # # # #
  3 # # # # # #
  4 # # # # # #
  5 # # # # # #
  6 # # # # # F
 > r 7 7
 Outside the board.

    1 2 3 4 5 6
  1 1 # # # # #
  2 # # # # # #
  3 # # # # # #
  4 # # # # # #
  5 # # # # # #
  6 # # # # # F
 >
```

Identifying Bugs and fixing:

1.

```python
def neighbors(self, r, c):
    for dr in (-1, 0, 1):
        for dc in (-1, 0, 1):
            if dr == 0 and dc == 0:
                continue
            nr, nc = r + dr, c + dc
            if 0 <= nr <= self.rows and 0 <= nc <= self.cols:
                yield nr, nc
```

Prompt:
I am working on a Python Minesweeper assignment. Please inspect the following Board class and fix the boundary-safe neighbour traversal bug.

In the neighbors() method, the current condition incorrectly permits row or column indices equal to self.rows or self.cols. This is a zero-based grid, so valid indices must be strictly less than these dimensions.

Requirements:

1. Fix the boundary condition in neighbors().
2. Preserve the existing class structure, method signatures, and coding style.
3. Do not rewrite unrelated methods or add unnecessary dependencies.
4. Explain the bug and the exact change.
5. Provide a small Python test that verifies neighbour coordinates are always in bounds, including for all four corners and edge cells of a 6 × 6 board.
6. Do not claim the tests pass unless they have actually been executed.

```python
    def neighbors(self, r, c):
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                if dr == 0 and dc == 0:
                    continue
                nr, nc = r + dr, c + dc
                if 0 <= nr < self.rows and 0 <= nc < self.cols:
                    yield nr, nc

```

```
PS C:\Users\233285\Downloads\LAB\SE Lab\Solo\13_minesweeper> python test_board.py
All boundary tests passed!
```

2.

```
def reveal(self, start):
    stack = [start]
    hit_mine = False
    while stack:
        pos = stack.pop()
        if pos in self.revealed or pos in self.flags:
            continue
        r, c = pos
        self.revealed.add(pos)
        if pos in self.mines:
            hit_mine = True
            continue
        if self.adjacent_mines(r, c) == 0:
            stack.extend(n for n in self.neighbors(r, c) if n not in self.revealed)
    return hit_mine

def toggle_flag(self, pos):
    if pos in self.revealed:
        return False
    if pos in self.flags:
        self.flags.remove(pos)
    else:
        self.flags.add(pos)
    return True

def won(self):
    return len(self.revealed) == self.rows * self.cols - self.mine_total
```

Fix the win detection and flag behaviour in my existing Python Minesweeper project.

Requirements:

1. Ensure a flagged cell cannot be revealed while it remains flagged.
2. Ensure flagging a cell twice removes the flag.
3. Ensure the game is won only when all non-mine cells have been revealed.
4. Revealing a mine must never result in a win.
5. Preserve the existing code structure and command interface.
6. Avoid unrelated changes and unnecessary dependencies.

Explain the changes and provide tests for flagging, unflagging, revealing flagged cells, mine hits, and winning.

```
    def reveal(self, start):
        stack = [start]
        hit_mine = False
        while stack:
            pos = stack.pop()
            if pos in self.revealed or pos in self.flags:
                continue
            r, c = pos
            self.revealed.add(pos)
            if pos in self.mines:
                hit_mine = True
                continue
            if self.adjacent_mines(r, c) == 0:
                stack.extend(n for n in self.neighbors(r, c) if n not in self.revealed)
        return hit_mine

    def toggle_flag(self, pos):
        if pos in self.revealed:
            return False
        if pos in self.flags:
            self.flags.remove(pos)
        else:
            self.flags.add(pos)
        return True

    def won(self):
        safe_cells = self.rows * self.cols - self.mine_total
        revealed_safe_cells = len(self.revealed - self.mines)

        return (
            not (self.revealed & self.mines)
            and revealed_safe_cells == safe_cells
        )
```

```
PS C:\Users\233285\Downloads\LAB\SE Lab\Solo\13_minesweeper> python test_flags_win.py
All flag and win tests passed!
```

3.

```
class Minesweeper:
    def __init__(self):
        self.board = Board()
```

Modify my existing Python Minesweeper game to support three difficulty modes: Easy, Medium, and Hard.

Requirements:

1. Easy: 6 rows, 6 columns, 6 mines.
2. Medium: 8 rows, 8 columns, 12 mines.

3. Hard: 10 rows, 10 columns, 20 mines.
4. Ask the player to select a difficulty when the game starts.
5. Create the Board using the selected dimensions and mine count.
6. Keep the existing reveal, flag, win detection, and input validation behaviour.
7. Update the board display so row and column labels remain correct for every board size.
8. Keep the code modular and avoid unnecessary dependencies.
9. Explain the code changes and provide tests for all three modes and invalid difficulty input.

```python
class Minesweeper:
    def __init__(self):
        self.difficulties = {
            "easy": (6, 6, 6),
            "medium": (8, 8, 12),
            "hard": (10, 10, 20),
        }

        while True:
            choice = input(
                "Choose difficulty (easy/medium/hard): "
            ).strip().lower()

            if choice in self.difficulties:
                break

            print(
                "Invalid difficulty. Please choose "
                "easy, medium, or hard."
            )

        rows, cols, mines = self.difficulties[choice]
        self.difficulty = choice
        self.board = Board(rows=rows, cols=cols, mines=mines)

```

```
PS C:\Users\233285\Downloads\LAB\SE Lab\Solo\13_minesweeper> python main.py
Choose difficulty (easy/medium/hard): expert
Invalid difficulty. Please choose easy, medium, or hard.
Choose difficulty (easy/medium/hard):
```

4.

```
28      def run(self):
29          if self.board.reveal((r, c)):
30              self.display(reveal_mines=True)
31              print("BOOM! You hit a mine.")
32              return
33          if self.board.won():
34              self.display()
35              print("You cleared the board!")
36              return
```

Update my existing Python Minesweeper game to provide concise feedback for each player reveal action.

Requirements:

1. Print feedback when the player reveals a safe cell, such as the number of adjacent mines.
2. If the reveal triggers a flood-fill, print feedback only once for the player's action, not for every cell revealed internally.
3. Preserve the existing mine-hit and win messages.
4. Do not print reveal feedback for flagging or invalid commands.
5. Keep the existing modular structure and make minimal changes.
6. Provide tests or manual steps to verify safe reveals, flood-fill, mine hits, and winning.

```
PS C:\Users\233285\Downloads\LAB\SE Lab\Solo\13_minesweeper> python main.py
Choose difficulty (easy/medium/hard): easy
Minesweeper
Difficulty: Easy
Commands: r row col | f row col | q

   1 2 3 4 5 6
 1 # # # # # #
 2 # # # # # #
 3 # # # # # #
 4 # # # # # #
 5 # # # # # #
 6 # # # # # #
>
```